

Webinar: How RVO Health keep docs in Backstage up to date with Swimm [Save your spot →](#)

On this page:[Background](#)[A Repo from Scratch](#)[Plumbing vs porcelain commands](#)[Creating objects](#)[Commit objects](#)[Working with branches — under the hood](#)[Summary](#)**All posts**

INDUSTRY

A hands-on intro to Git internals: creating a repo from scratch

[Omer Rosenbaum](#) | 13 Sep 2020 | 15 min read

swimm

Deep Dive into Git Internals A Hands-On Tutorial

By Omer Rosenbaum



**On this page:****On this page:**

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



Using git on a daily basis, many times we fail to understand the ins and outs of how git internals really work – what is stored between commits, how is a diff encoded, what really happens when we use `git init` or how version control works (drum roll... 🥁), **we will create a repo** from scratch and see the wonders of **the plumbing beneath porcelain**.

This is our second post diving into the internals of git. If you missed our previous post, you can find it [here](#).

Background

In [our previous post](#) we covered the basic git objects — **blobs**, **trees**, and **commits**. We explained that a **blob** holds the contents of a file. A **tree** is a directory-listing, containing **blobs** and/or sub-**trees**. A **commit** is a snapshot of our working directory, with some meta-data such as the time or the commit message. Additionally, we discussed **branches** and how they are implemented, as they are nothing but a named reference to a **commit**. (These really show why Git is valuable for version control).

So far we set the ground covering fundamentals, and now we're ready to really Git going.

A Repo from Scratch

In order to deeply understand how git internals work, we will create a **repository**, but this time — build it from scratch.

We won't use `git init`, `git add` or `git commit` which will enable us to get a better hands-on understanding in the process.

Setting Up .git

Note — most posts with shell commands show UNIX commands. As in [our previous post](#), I will provide commands for both Windows and UNIX, with screenshots from Windows, for the sake of variance. When the commands are exactly the same, I will provide them only once.

Let's create a new working directory (also named "working tree"), and run `git status` within it:

```
C:\Users\omerr\swimminar>mkdir repo_2
C:\Users\omerr\swimminar>cd repo_2
C:\Users\omerr\swimminar\repo_2>git status
fatal: not a git repository (or any of the parent directories): .git
```

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



Alright, so `git` seems unhappy as we don't have a `.git` folder. The natural thing to do would be to simply create that directory:

```
C:\Users\omerr\swimminar\repo_2>mkdir .git
C:\Users\omerr\swimminar\repo_2>git status
fatal: not a git repository (or any of the parent directories): .git
```

Apparently, creating a `.git` directory is just not enough. We need to add some content to that directory.

A git repository has mainly two components:

1. A collection of objects — **blobs**, **trees**, and **commits**.
2. A system of naming those objects — called **references**.

A **repository** may also contain other things, such as git hooks, but at the very least — it must include objects and references.

Let's create a directory for the objects at `.git\objects` and a directory for the references (in short: **refs**) at `.git\refs` (on UNIX -based systems — `.git/objects` and `.git/refs` , respectively).

```
C:\Users\omerr\swimminar\repo_2>mkdir .git\objects
C:\Users\omerr\swimminar\repo_2>mkdir .git\refs
C:\Users\omerr\swimminar\repo_2>tree /f .git
Folder PATH listing for volume Windows
Volume serial number is 8E86-766C
C:\USERS\OMERR\SWIMMINAR\REPO_2\.GIT
├──objects
└──refs
```

One type of reference is **branches**. Internally, `git` calls **branches** by the name of **heads**. So we will create a directory for them — `.git\refs\heads`.

```
C:\Users\omerr\swimminar\repo_2>mkdir .git\refs\heads
C:\Users\omerr\swimminar\repo_2>tree /f .git
Folder PATH listing for volume Windows
Volume serial number is 8E86-766C
C:\USERS\OMERR\SWIMMINAR\REPO_2\.GIT
├──objects
├──refs
│   └──heads
```

This still doesn't change `git status` :

```
C:\Users\omerr\swimminar\repo_2>git status
fatal: not a git repository (or any of the parent directories): .git
```

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



How does `git` know where to start when looking for a **commit** in the **repository**?

As explained in [our previous post](#), it looks for `HEAD`, which points to the current active branch (or **commit**, in some cases). So, we need to create `HEAD`, which is just a file residing at `.git\HEAD`. We can apply the following:

On Windows: >

```
echo ref: refs/heads/master > .git\HEAD
```

On UNIX:

```
$ echo "ref: refs/heads/master" > .git/HEAD
```

★ So we now know how `HEAD` is implemented — it's simply a file, and its file contents describe what it points to.

Following the command above, `git status` seems to change its mind:

```
C:\Users\omerr\swimminar\repo_2>echo ref: refs/heads/master > .git\HEAD
C:\Users\omerr\swimminar\repo_2>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Notice that `git` believes we are on a branch called `master`, even though we haven't created this branch. As mentioned in [the previous post](#), `master` is just a name. We could also make `git` believe we are on a branch called `banana` if we wanted to:

```
C:\Users\omerr\swimminar\repo_2>echo ref: refs/heads/banana > .git\HEAD
C:\Users\omerr\swimminar\repo_2>git status
On branch banana

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

We will switch back to `master` for the rest of this post, just to adhere to the normal convention.

Now that we have our `.git` directory ready, can we work our way to make a **commit** (again, without using `git add` or `git commit`).

Plumbing vs porcelain commands

On this page:

[Background](#)[A Repo from Scratch](#)[Plumbing vs porcelain commands](#)[Creating objects](#)[Commit objects](#)[Working with branches — under the hood](#)[Summary](#)

At this point, it would be helpful to make a distinction between two types of `git` commands: **plumbing** and **porcelain**. The application of the terms oddly comes from toilets (yeah, these — ) , traditionally made of porcelain, and the infrastructure of plumbing (pipes and drains). We can say that the porcelain layer provides a user-friendly interface to the plumbing. Most people only deal with the porcelain. Yet, when things go (terribly) wrong, and as someone would want to understand why, they would have to roll-up their sleeves to check the plumbing. (Note: these terms are not mine, they are used very widely in `git`).

`git` uses this terminology in analogy, to separate the low-level commands that users don't usually need to use directly ("plumbing" commands) from the more user-friendly high level commands ("porcelain" commands).

So far, we have dealt with porcelain commands — `git init` , `git add` or `git commit` . Next, we transition to plumbing commands.

Creating objects

Starting with creating an object and writing it into the objects' database of `git` , residing within `.git\objects` , we can find out the SHA-1 hash value of a **blob** object by using our first plumbing command, `git hash-object` , in the following way:

On Windows:

```
> echo git is awesome | git hash-object --stdin
```

On UNIX:

```
$ echo "git is awesome" | git hash-object --stdin
```

By using `--stdin` we are instructing `git hash-object` to take its input from the standard input. This will provide us with the relevant **SHA-1 hash** value. In order to actually write that **blob** into `git` 's object database, we can simply add the `-w` switch for `git hash-object` . Then, we can check the contents of `.git` folder , and see that it changed.

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



We can now see the hash of our **blob** is —
54f6...36. We can also see that a directory has been created under `.git\objects`, a directory named `54`, and within it — a file by the name of `f6...36`. So `git` actually takes the first two characters of the of the **SHA-1** hash and uses them as the name of a working directory, and the remaining SHA-1 characters are used as the filename, for the file that actually contains the **blob** object.

Why is that so? Consider a fairly big repository, one that has 300,000 objects (**blobs**, **trees**, and **commits**) in its database. To look up a hash inside that list of 300,000 hashes can take a while. Thus, `git` simply divides that problem by 256. To look up the hash above, `git` would first look for the directory named `54` inside the director `.git\objects`, which may have up to 256 directories (`00` through `FF`). Then, it will search that working directory, narrowing down the search.

Back to our process of generating a **commit**. We have now created an object. What is the type of that object? We can use another plumbing command, `git cat-file -t` (`-t` stands for “type”), to check that out:

```
C:\Users\omerr\swimminar\repo_2>git cat-file -t 54f6e651fe2728e9972323dac32bf2192f17af36
blob
```

Not surprisingly, this object is a **blob**. We can also use `git cat-file -p` (`-p` stands for “pretty-print”) to see its contents:

```
C:\Users\omerr\swimminar\repo_2>git cat-file -p 54f6e651fe2728e9972323dac32bf2192f17af36
git is awesome
```

This process of creating a **blob** usually happens when we add something to the **staging area** — that is, when we use `git add`. Remember that `git` creates a **blob** of the *entire* file that is staged. Even if a single character is modified or added (as we added

! in our example in [the previous post](#)), the file has a new **blob** with a new **hash**.

Will there be any change to `git status` ?

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



```
C:\Users\omerr\swimminar\repo_2>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Apparently, no. Adding a **blob** object to `git`'s internal database doesn't change the status, as `git` doesn't know of any tracked or untracked files at this stage. We need to track this file — add it to the **staging area**. To do that, we can use the plumbing command `git update-index`, like so:

```
git update-index --add --cacheinfo 100644 <blob-hash> <filename>.
```

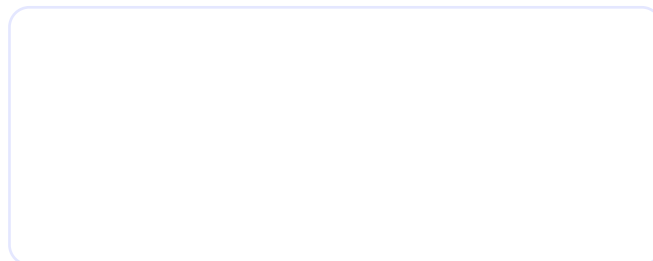
Note: (The `cacheinfo` is a 16-bit file mode [as stored by git](#), following the layout of [POSIX types and modes](#). This is not within the scope of this post).

Running the command above will result in a change to `.git`'s contents:



Can you spot the change? A new file by the name of `index` was created. This is it — the famous **index** (or **staging area**), is basically a file that resides within `.git\index`.

So now that our **blob** has been added to the **index**, we expect `git status` to look differently:



That's interesting! Two things happened here.

First, we can see that `new_file.txt` appears in green, in the `Changes to be committed` area. That is so because the **index** now has `new_file.txt`, waiting to be committed.

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



Second, we can see that `new_file.txt` appears in red — because `git` believes the *file* `my_file.txt` has been deleted, and the fact that the file has been deleted is not staged. This happens as we added the **blob** with the contents `git is awesome` to the objects' database, and told the **index** that the new file `my_file.txt` has the contents of that **blob**, but we never actually created that file. This can be easily solved by taking the contents of the **blob**, and writing them to our file system, to a file called `my_file.txt` :



As a result, it will no longer appear in red by `git status` :

```
C:\Users\omerr\swimminar\repo_2>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   my_file.txt
```

Commit objects

So now it's time to create a **commit** object from our **staging area**. As explained in [our previous post](#), a **commit** object has a reference to a **tree**, so we need to create a **tree**. We can do it with the command `git write-tree`, which records the contents of the **index** in a **tree** object. Of course, we can use `git cat-file -t` to see that it's indeed a **tree**:

```
C:\Users\omerr\swimminar\repo_2>git write-tree
de8bfa8ec703584964466eb9c830ee47152b8ab0
C:\Users\omerr\swimminar\repo_2>git cat-file -t de8bfa8ec703584964466eb9c830ee47152b8ab0
tree
```

And we can use `git cat-file -p` to see its contents:

```
C:\Users\omerr\swimminar\repo_2>git cat-file -p de8bfa8ec703584964466eb9c830ee47152b8ab0
100644 blob 54f6e651fe2728e9972323dac32bf2192f17af36    my_file.txt
```

Great, so we created a **tree**, and now we need to create a **commit** object **and add a commit message** that references this **tree**. To do that, we

can use

```
git commit-tree <tree-hash> -m <commit message> :
```

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



You should now feel comfortable with the commands used to check the created object's type, and print its contents:

Note that this **commit** doesn't have a **parent**, because it's the first **commit**. When we add another **commit** we will have to declare its **parent** — we will do so later.

The last hash that we got — `80e...8f`, is a **commit**'s hash. We are actually very used to using these hashes — we look at them all the time. Note that this **commit** owns a **tree** object, with its own hash, which we rarely specify explicitly.

Will something change in `git status` ?

Nope 🤔.

Why is that? Well, to know that our file has been committed, `git` needs to know about the latest **commit**. How does `git` do that? It goes to the `HEAD` :

`HEAD` points to `master`, but what is `master` branch ? We haven't really created it yet. As we explained earlier in this post, a branch is simply a named reference to a **commit**. And in this case, we would

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood ⭐ In sum, a **branch** is — just a file inside .

Summary



like `master` to refer to the **commit** with the hash `80e8ed4fb0bfc3e7ba88ec417ecf2f6e6324998f` . We can achieve this by simply creating a file at `\refs\heads\master` , with the contents of this hash, like so:

`git\refs\heads` , containing a hash of the **commit** it refers to.

Now, finally, `git status` and `git log` seem to appreciate our efforts:



We have successfully created a **commit** without using porcelain commands! How cool is that? 🎉

Working with branches — under the hood

Just as we've created a **repository** and a **commit** without using `git init` , `git add` or `git commit` , now we will create and switch between **branches** without using porcelain commands (`git branch` or `git checkout`). It's perfectly understandable if you are excited, I am too 😊

Let's start:

So far we only have one **branch**, named `master` . To create another one with the name of `test` (as the equivalent of `git branch test`), we would need to simply create a file named `test` within `git\refs\heads` , and the contents of that file would be the same **commit**'s hash as the one `master` points to.

```

C:\Users\omerr\swimmar\repo_2>echo 80e8ed4fb0bfc3e7ba88ec417ecf2f6e6324998f > .git\refs\heads\test
C:\Users\omerr\swimmar\repo_2>tree /f .git

```

How it works	Chat	Document	Product tour	Integrations	Modernize legacy code	Maintain legacy code	Documentation platform
--------------------	------	----------	-----------------	--------------	-----------------------------	----------------------------	---------------------------



Product ▾

Use
Cases ▾

R

Let's also switch to our newly created branch (the equivalent of `git checkout test`). For that, we should change `HEAD` to point to our new branch:

As we can see, both `git status` and `git log` confirm that `HEAD` now points to `test`, which is, therefore, the active branch.

We can now use the commands we have already used to create another file and add it to the index:

Using the commands above, we have created a file named `test.txt`, with the content of `Testing`, created a corresponding **blob**, and added it to the **index**. We also created a **tree object** representing the **index**.

It's now time to create a **commit** referencing this **tree**. This time, we should also specify the *parent* of this **commit** — which would be the previous **commit**. We specify the parent using the `-p` switch of `git commit-tree` :

We have just created a **commit**, with a **tree** object as well as a parent, as we can see:

```
C:\Users\omerr\swimminar\repo_2>git cat-file -p 465ef9eacc72378c786e4685fd7dff6b068a005e
tree b1c20705f1d04ad233df7c02e322f22dbd24093f
parent 80e8ed4fb0bfc3e7ba88ec417ecf2f6e6324998f
author omerr <omer@swimm.io> 1587225717 +0300
committer omerr <omer@swimm.io> 1587225717 +0300

Testing
```

On this page:

Background

A Repo from Scratch

Plumbing vs porcelain commands

Creating objects

Commit objects

Working with branches — under the hood

Summary



Will `git log` show us the new **commit**?

```
C:\Users\omerr\swimminar\repo_2>git log
commit 80e8ed4fb0bfc3e7ba88ec417ecf2f6e6324998f (HEAD -> test, master)
Author: omerr <omer@swimm.io>
Date: Sat Apr 18 18:35:34 2020 +0300

First message
```

As we can see, `git log` doesn't show anything new. Why is that? 😞 Remember that `git log` traces the **branches** to find relevant commits to show. It shows us now `test` and the **commit** it points to, and it also shows `master` which points to the same **commit**. That's right — we need to change `test` to point to our new **commit**. We can do that by simply changing the contents of `.git\refs\heads\test` :

```
C:\Users\omerr\swimminar\repo_2>echo 465ef9eacc72378c786e4685fd7dff6b068a005e > .git\refs\heads\test
C:\Users\omerr\swimminar\repo_2>git log
commit 465ef9eacc72378c786e4685fd7dff6b068a005e (HEAD -> test)
Author: omerr <omer@swimm.io>
Date: Sat Apr 18 19:01:57 2020 +0300

Testing

commit 80e8ed4fb0bfc3e7ba88ec417ecf2f6e6324998f (master)
Author: omerr <omer@swimm.io>
Date: Sat Apr 18 18:35:34 2020 +0300

First message
```

`git log` goes to `HEAD` , which tells it to go to the branch `test` , which points to **commit** `465...5e` , which links back to its parent **commit** `80e...8f` .

Feel free to admire the beauty, we *git* you 😊

Summary

After setting the grounds in [the first post](#) with elementary terminology and `git` internal know-how, in this post, we fearlessly deep-dived into `git` ; We stopped using porcelain commands and switched to plumbing commands. By using `echo` and low-level commands such as `git hash-object` , we were able to create a **blob**, add it to the **index**, create a **tree** of the **index**, and create a **commit** object pointing to that **tree**. We were also able to create and switch between **branches**. Kudos to those of you who tried this on their own! 🍌

Disclaimer: This post and [the previous one](#) on `git` internals, are part of our new [Swimminars blog](#)

[series](#) (read more [here](#) to find out what Swimminars are). We plan to provide similar posts in the future, so please **comment** and let us know your thoughts or questions on your journey into `git internals`.

On this page:

- Background
- A Repo from Scratch
- Plumbing vs porcelain commands
- Creating objects
- Commit objects
- Working with branches — under the hood
- Summary



Omer Rosenbaum, Swimm's Chief Technology Officer. Cyber training expert and Founder of Checkpoint Security Academy. Author of [Computer Networks \(in Hebrew\)](#). Visit My [YouTube Channel](#).

...

Additional references

A lot has been written and said about `git`. Specifically, I found these references to be useful:

- [Tim Berglund's lecture — "Git From the Bits Up"](#)
- [Git from the Bottom Up — by John Wiegley](#)
- [as promised, docs: git for the confused](#)
- [Git Internals — Git Objects — from Pro Git book, by Scott Chacon and Ben Straub](#)

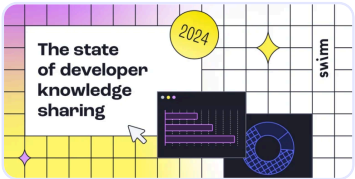


Omer Rosenbaum | CTO & Co-founder

Omer founded the Check Point Security Academy and was the Cyber Security Lead at ITC, an educational organization that trains talented professionals to develop careers in technology. Omer has a MA in Linguistics from Tel Aviv University and is the creator behind the Brief YouTube Channel.

Keep reading

[All posts](#)



INDUSTRY



INDUSTRY

Contextualizing AI: Source code

Omer Rosenbaum | Jan 8, 24

| 6 min read

AI coding assistants have ushered in a new era of software development. Th...

Nearly every developer (97%) is either using or experimenting with AI tool...

A Report from the Secretariat

Plumbing vs. porcelain commands

experiment

Commit objects

Working with branches — under the hood

Summary



Contact sales

Get in touch with a
Swimm expert to
discuss your needs.

See an in-depth
overview of Swimm.

Pick a time

Watch now

Company

About us

Careers

Community

Mermaid.js

Integrations